

HDOC DAY 30

AIM: To write a menu-driven C program that performs operations like Bubble Sort, Binary Search, Insertion, Deletion, and Right Rotation on an array.

ALGORITHM:

1. **Start** the program and ask the user to enter the size of the array and its elements.
2. Display a menu with the following choices:
 - a. Choice 1: Sort using Bubble Sort and search using Binary Search.
 - b. Choice 2: Insert or Delete an element at a specific position.
 - c. Choice 3: Rotate array elements to the right by k positions.
 - d. Choice 4: Exit.
3. Read the choice from the user.
4. **If Choice = 1:**
 - a. Sort the array using two loops (Bubble Sort).
 - b. Print the sorted array.
 - c. Ask the user for a search element and perform Binary Search using low, high, and mid pointers.
5. **If Choice = 2:**
 - a. Ask whether the user wants to Insert or Delete.
 - b. For **Insertion**: Take position and element, shift elements to the right, and put the new element at the position.
 - c. For **Deletion**: Take position, shift elements to the left, and decrease array size.
6. **If Choice = 3:**
 - a. Take number of rotations k.
 - b. Shift array elements to the right by 1 position k times.
7. **If Choice = 4:**
 - a. Exit the program.
8. Repeat the menu until the user selects option 4.

C CODE :

```
#include <stdio.h>
```

```
int main() {  
    int arr[100];  
    int n, choice, i, j, temp;  
  
    printf("Enter number of elements in array: ");  
    scanf("%d", &n);  
  
    printf("Enter %d elements:\n", n);  
    for(i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    while(1) {  
        printf("\n--- MENU ---\n");  
        printf("1. Sort using Bubble Sort & Binary Search\n");  
        printf("2. Insert/Delete at a position\n");  
        printf("3. Rotate right by k positions\n");  
        printf("4. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        if(choice == 1) {  
            // Bubble Sort  
            for(i = 0; i < n - 1; i++) {  
                for(j = 0; j < n - i - 1; j++) {  
                    if(arr[j] > arr[j+1]) {
```

```
        temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
    }
}
}
```

```
printf("\nSorted Array: ");
for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
```

```
// Binary Search
```

```
int key, low = 0, high = n - 1, mid, found = 0;
printf("Enter element to search: ");
scanf("%d", &key);
```

```
while(low <= high) {
    mid = (low + high) / 2;
    if(arr[mid] == key) {
        printf("Element found at position %d (index %d)\n", mid + 1, mid);
        found = 1;
        break;
    }
    else if(arr[mid] < key) {
```

```

        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

if(found == 0) {
    printf("Element not found!\n");
}
}

else if(choice == 2) {
    int subChoice;

    printf("1. Insert Element\n2. Delete Element\nEnter choice: ");
    scanf("%d", &subChoice);

    if(subChoice == 1) {
        int pos, val;

        printf("Enter position to insert (1 to %d): ", n + 1);
        scanf("%d", &pos);
        printf("Enter value to insert: ");
        scanf("%d", &val);

        if(pos < 1 || pos > n + 1) {
            printf("Invalid Position!\n");
        } else {

```

```

        for(i = n; i >= pos; i--) {
            arr[i] = arr[i - 1];
        }
        arr[pos - 1] = val;
        n++;
        printf("Array after insertion: ");
        for(i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}

else if(subChoice == 2) {
    int pos;
    printf("Enter position to delete (1 to %d): ", n);
    scanf("%d", &pos);

    if(pos < 1 || pos > n) {
        printf("Invalid Position!\n");
    } else {
        for(i = pos - 1; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;
        printf("Array after deletion: ");
        for(i = 0; i < n; i++) {

```

```

        printf("%d ", arr[i]);
    }
    printf("\n");
}
}
}

else if(choice == 3){
    int k;

    printf("Enter k (number of rotations): ");
    scanf("%d", &k);

    k = k % n; // to handle if k is greater than n

    for(i = 0; i < k; i++) {
        int last = arr[n - 1];
        for(j = n - 1; j > 0; j--) {
            arr[j] = arr[j - 1];
        }
        arr[0] = last;
    }

    printf("Array after right rotation: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```
}  
else if(choice == 4){  
    printf("Exiting program...\n");  
    break;  
}  
else {  
    printf("Invalid choice! Try again.\n");  
}  
}  
  
return 0;  
}
```

CODE SCREENSHOT :

```

#include <stdio.h>

int main() {
    int arr[100];
    int n, choice, i, j, temp;

    printf("Enter number of elements in array: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    while(1) {
        printf("\n--- MENU ---\n");
        printf("1. Sort using Bubble Sort & Binary Search\n");
        printf("2. Insert/Delete at a position\n");
        printf("3. Rotate right by k positions\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if(choice == 1) {
            // Bubble Sort
            for(i = 0; i < n - 1; i++) {
                for(j = 0; j < n - i - 1; j++) {
                    if(arr[j] > arr[j+1]) {
                        temp = arr[j];
                        arr[j] = arr[j+1];
                        arr[j+1] = temp;
                    }
                }
            }

            printf("\nSorted Array: ");
            for(i = 0; i < n; i++) {
                printf("%d ", arr[i]);
            }
            printf("\n");

            // Binary Search
            int key, low = 0, high = n - 1, mid, found = 0;
            printf("Enter element to search: ");
            scanf("%d", &key);

            while(low <= high) {
                mid = (low + high) / 2;
                if(arr[mid] == key) {
                    printf("Element found at position %d (index %d)\n", mid + 1, mid);
                    found = 1;
                    break;
                }
                else if(arr[mid] < key) {
                    low = mid + 1;

```

```

                }
                else {
                    high = mid - 1;
                }
            }

            if(found == 0) {
                printf("Element not found!\n");
            }
        }
        else if(choice == 2) {
            int subChoice;
            printf("1. Insert Element\n2. Delete Element\nEnter choice: ");
            scanf("%d", &subChoice);

            if(subChoice == 1) {
                int pos, val;
                printf("Enter position to insert (1 to %d): ", n + 1);
                scanf("%d", &pos);
                printf("Enter value to insert: ");
                scanf("%d", &val);

                if(pos < 1 || pos > n + 1) {
                    printf("Invalid Position!\n");
                }
                else {
                    for(i = n; i >= pos; i--) {
                        arr[i] = arr[i - 1];
                    }

```

```

        arr[pos - 1] = val;
        n++;
        printf("Array after insertion: ");
        for(i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}
else if(subChoice == 2) {
    int pos;
    printf("Enter position to delete (1 to %d): ", n);
    scanf("%d", &pos);

    if(pos < 1 || pos > n) {
        printf("Invalid Position!\n");
    } else {
        for(i = pos - 1; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;
        printf("Array after deletion: ");
        for(i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}
}

```

```

}
else if(choice == 3) {
    int k;
    printf("Enter k (number of rotations): ");
    scanf("%d", &k);

    k = k % n; // to handle if k is greater than n

    for(i = 0; i < k; i++) {
        int last = arr[n - 1];
        for(j = n - 1; j > 0; j--) {
            arr[j] = arr[j - 1];
        }
        arr[0] = last;
    }

    printf("Array after right rotation: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
else if(choice == 4) {
    printf("Exiting program...\n");
    break;
}
else {
    printf("Invalid choice! Try again.\n");
}
}

```

```
        printf("Invalid choice! Try again.\n");  
    }  
}  
  
return 0;  
}
```

OUTPUT SCREENSHOT;

```
[manalithakar@Manalis-MacBook-Air ~ % gcc 30.c -o 30
[manalithakar@Manalis-MacBook-Air ~ % ./30
Enter number of elements in array: 5
Enter 5 elements:
40 10 50 20 30
```

```
--- MENU ---
1. Sort using Bubble Sort & Binary Search
2. Insert/Delete at a position
3. Rotate right by k positions
4. Exit
Enter your choice: 1
```

```
Sorted Array: 10 20 30 40 50
Enter element to search: 30
Element found at position 3 (index 2)
```

```
--- MENU ---
1. Sort using Bubble Sort & Binary Search
2. Insert/Delete at a position
3. Rotate right by k positions
4. Exit
Enter your choice: 2
1. Insert Element
2. Delete Element
Enter choice: 1
Enter position to insert (1 to 6): 3
Enter value to insert: 25
Array after insertion: 10 20 25 30 40 50
```

```
--- MENU ---
1. Sort using Bubble Sort & Binary Search
2. Insert/Delete at a position
3. Rotate right by k positions
4. Exit
Enter your choice: 2
1. Insert Element
2. Delete Element
Enter choice: 2
Enter position to delete (1 to 6): 4
Array after deletion: 10 20 25 40 50
```

```
--- MENU ---
1. Sort using Bubble Sort & Binary Search
2. Insert/Delete at a position
3. Rotate right by k positions
4. Exit
Enter your choice: 3
Enter k (number of rotations): 2
Array after right rotation: 40 50 10 20 25
```

```
--- MENU ---
1. Sort using Bubble Sort & Binary Search
2. Insert/Delete at a position
3. Rotate right by k positions
4. Exit
Enter your choice: 4
Exiting program...
manalithakar@Manalis-MacBook-Air ~ %
```